

The Opus Audio Codec: Theory and Implementation

Jack Geraghty

Contents

1	Introduction	2
1.1	The Reference Implementation	2
1.2	Target Audience and Prerequisites	2
1.3	Scope and Organization	3
1.4	Notation Conventions	3
2	Architecture Overview	3
2.1	Two Codecs, One Bitstream	3
2.2	The Layered Stack	4
2.3	Decoding a Packet: An Overview	4
2.4	Sample Rates and the 48 kHz Reference	5
2.5	From Quantities to Symbols	6
3	The Range Coder	6
3.1	Why Entropy Coding	6
3.2	The Integer Range Coder	7
3.2.1	Decoding a symbol	7
3.2.2	The renormalization loop	8
3.2.3	Encoding a symbol	8
3.3	Carry Propagation	8
3.4	The ICDF Representation	9
3.5	Binary Symbols and Raw Bits	10
3.6	Bit Budgeting: tell() and tell_frac()	11
3.7	The Final-Range Oracle	12
4	Packet Framing	12
4.1	The Self-Delimiting Question	12
4.2	The TOC Byte	13
4.2.1	The 32 configurations	13
4.3	Frame-Packing Codes	13
4.3.1	Code 3 in detail	15

4.4	Validity Rules	15
5	CELT: Transform Coding	15
5.1	The MDCT and Lapped Transforms	16
5.1.1	The window and perfect reconstruction	16
5.1.2	Short blocks for transients	17
5.1.3	Computing the MDCT through an FFT	17
5.2	The Energy Envelope	17
5.2.1	Coarse energy and two-dimensional prediction	18
5.3	Pyramid Vector Quantization	18
5.3.1	The pulse search	19
5.4	CWRS: Enumerating the Codebook	19
6	Python API Reference	20
6.1	Introduction	20

1 Introduction

This manual provides a comprehensive overview of the algorithms, optimizations, and design decisions that make up the Opus audio codec [5]. The focus is on understanding both the theoretical foundations and the practical implementations of the techniques Opus combines: the information theory of its entropy coder, the linear prediction of its speech path, the transform coding of its music path, and the fixed-point and SIMD machinery that lets all of it run in real time.

The material emphasizes the *why* and *how* of each component: *why* certain design choices matter, and *how* they are implemented efficiently. It covers both the mathematics and the practical computational considerations that make these methods work in production systems.

Opus is the IETF’s general-purpose, royalty-free audio codec, standardized in 2012 and updated by RFC 8251 [4]. It is the default audio codec of WebRTC and is widely deployed in streaming and conferencing. Its defining characteristic is that it is not one codec but two, combined behind a single bitstream. A linear-predictive speech coder (*SILK*) and a transform-domain music coder (*CELT*, the Constrained-Energy Lapped Transform) share one entropy coder, one packet format, and one container. The encoder selects, per frame, between SILK, CELT, or both together (the *hybrid* mode). The combination spans a range no single technique covers well: 6 kb/s narrowband speech to 510 kb/s fullband stereo music, frame sizes from 2.5 ms to 60 ms, and sample rates from 8 kHz to 48 kHz, all decoded by one receiver that is not told in advance which mode each packet uses.

1.1 The Reference Implementation

Throughout, the implementation under discussion is `opus_native`, a pure-Rust, dependency-free realization of RFC 6716. Every code listing in this manual is drawn from that source. Three of its design choices shape the discussion:

- **Bit-exact by construction.** The bitstream is defined by the range coder (Section 3), not by any one encoder’s choices. The implementation’s entropy arithmetic follows the RFC byte-for-byte, and the decoder is validated against the official test vectors on the *final-range oracle*, a per-packet checksum of the entropy-coder state that must agree with the reference to the bit.
- **Safe by default.** The crate denies `unsafe` everywhere except a handful of explicitly annotated SIMD kernels (??). The signal-processing math is ordinary safe Rust; the only machine intrinsics sit in encoder hot loops where a faster, approximate result is provably harmless (??).
- **Decode-first.** The decoder is the normative half of the specification and the half the conformance vectors exercise. The encoder is free to make any choice the decoder can follow; this asymmetry recurs throughout the manual and is the key to several optimizations.

1.2 Target Audience and Prerequisites

This material assumes familiarity with:

- Discrete signals, sampling, and the discrete Fourier transform
- Linear algebra (vectors, matrices, inner products, projection)
- Basic probability and the idea of entropy / information content
- Programming and algorithm analysis; the listings are in Rust, but no Rust expertise is required to follow them

No prior knowledge of speech coding, transform coding, arithmetic coding, or psychoacoustics is assumed; each is introduced where it is first needed. A reader who has worked through a companion treatment of FFT-based spectral analysis will find the transform-coding sections (CELT) immediately familiar, and the linear-prediction sections (SILK) a complementary story told in the time domain.

1.3 Scope and Organization

The manual is organized bottom-up, from the bit to the file, mirroring the layering of the codec itself:

1. **Architecture Overview** (Section 2) — the two codecs, three modes, the shared range coder, and the sample-rate model.
2. **The Range Coder** (Section 3) — entropy coding, arithmetic/range coding, the ICDF representation, raw bits, `tell()`, and the final-range oracle.
3. **Packet Framing** (Section 4) — the TOC byte, the 32 configurations, frame packing codes 0–3, padding, and validity rules.
4. **CELT: Transform Coding** (Section 5) — the MDCT, the Vorbis window and TDAC, pyramid vector quantization, CWRS enumeration, band splitting, energy coding, bit allocation, and the encoder’s analysis.
5. **SILK: Linear-Predictive Coding** (??) — linear prediction, LSF/NLSF quantization, long-term (pitch) prediction, the noise-shaping quantizer, gains, the shell coder, and resampling.
6. **The Opus Layer** (??) — mode dispatch, the SILK/CELT crossover, redundancy at transitions, packet-loss concealment, in-band FEC, and discontinuous transmission.
7. **Ogg Encapsulation** (??) — pages, the Ogg CRC, lacing, packet reassembly, `OpusHead/OpusTags`, and granule timing.
8. **Multistream and Surround** (??) — coupled and uncoupled streams, channel mapping families.
9. **Computational Optimizations** (??) — SIMD dot products, the SIMD PVQ search, the MDCT pre-rotation, the FFT backend seam, table-driven CWRS, and buffer reuse.
10. **Numerical Considerations and Conformance** (??) — fixed-point versus float, bit-exactness, the oracle, and the conformance criterion.

1.4 Notation Conventions

Throughout this manual:

- $x[n]$ denotes a time-domain sample at index n ; $X[k]$ a frequency-domain coefficient at bin k
- f_s denotes a sample rate in Hz; Opus’s internal reference rate is always 48 kHz
- N denotes a transform length or vector dimension; K a pulse count
- LM (“log multiplier”) denotes $\log_2$ of the frame-size multiplier: $\text{LM} = 0, 1, 2, 3$ for 2.5, 5, 10, 20 ms CELT frames
- The range coder’s state is the pair (val, rng)
- Qn denotes a fixed-point value scaled by 2^n : a Q15 value v represents the real number $v/2^{15}$
- $\text{sgn}(x)$ is the sign of x ; $\lfloor \cdot \rfloor$ and $\lceil \cdot \rceil$ are floor and ceiling
- Section references like “§4.3.4” without further qualification refer to RFC 6716 [5]

2 Architecture Overview

This section describes the codec as a whole: its components, the path a packet takes through them, and the design decisions that the rest of the manual elaborates. Later sections treat each layer in detail.

2.1 Two Codecs, One Bitstream

Opus is built on the observation that audio divides into two regimes that reward opposite coding strategies. The codec implements both and selects between them per frame.

Speech versus Music: Two Coding Strategies

Speech is produced by a physical system (the vocal folds exciting the vocal tract) that is well modeled as a slowly varying *linear filter* driven by a sparse *excitation*. Coding speech efficiently means coding that model: estimate the filter, estimate the pitch, and transmit only the residual that the model fails to predict. This is **linear-predictive coding** (LPC), and it is the approach SILK takes, in the *time domain*, at internal rates of 8, 12, or 16 kHz.

Music is arbitrary: dense, polyphonic, and percussive, with energy spread across the full 20 kHz audible band and no simple source model. Coding music efficiently means working in the *frequency domain*, where the ear’s own frequency analysis allows bits to be spent where they are audible and withheld where masking hides them. This is **transform coding**, and it is the approach CELT takes, via the Modified Discrete Cosine Transform (MDCT) at 48 kHz.

Neither technique performs both tasks well: LPC degrades on polyphonic material, and transform coding spends bits inefficiently on the strong predictable structure of speech. Opus therefore retains both and switches between them per frame; in the overlap region (wideband speech with a musical background) it runs them together as the **hybrid** mode.

This yields the three **modes**, selected by the packet header (Section 4) and dispatched by the decoder (??):

Mode	Coder(s)	Typical use
SILK-only	SILK	low-bitrate speech, 8–16 kHz audio band
Hybrid	SILK (low) + CELT (high)	wideband speech + full bandwidth
CELT-only	CELT	music, low delay, high bitrate

In hybrid mode the split is by frequency: SILK codes the signal up to roughly 8 kHz (it runs at a 16 kHz internal rate), and CELT codes the band above, starting at MDCT band 17 (??). Both coders write into the same range coder, so there is no boundary marker between them in the byte stream: the CELT data is simply a continuation of the same range-coded message that began with the SILK data.

2.2 The Layered Stack

Table 1 lists the layers as the implementation modularizes them, bottom to top. The lower layers (range coder, packet, Ogg) are pure framing and entropy: exact integer arithmetic, `no_std`-compatible, with no audio math. The middle layers (SILK, CELT) perform the signal processing. The top layer (the Opus decoder and encoder) provides the glue: mode dispatch, resampling, and the cross-frame state used to handle mode transitions.

Layer	Specification	Responsibility
ogg	RFC 3533 / 7845	file container, pages, timing
multistream	RFC 7845 §5.1.1	surround: N coupled/mono streams
decoder/encoder	RFC 6716 §4/§5	mode dispatch, transitions, PLC/FEC/DTX
silk	RFC 6716 §4.2	linear-predictive speech coder
celt	RFC 6716 §4.3	MDCT transform coder
packet	RFC 6716 §3	TOC byte, frame packing
range	RFC 6716 §4.1/§5.1	entropy coder

Table 1: The Opus layer stack, bottom to top.

2.3 Decoding a Packet: An Overview

To decode one Opus packet, the decoder performs the following steps:

1. Reads the first byte, the **TOC** (Table of Contents), which names the mode, audio bandwidth, frame size, and channel count (Section 4).
2. Splits the remaining bytes into 1–48 *frames* according to the TOC’s frame-packing code.
3. For each frame, constructs a range decoder over its bytes and, depending on the mode, runs the SILK decoder, the CELT decoder, or both into a shared range decoder.
4. Resamples the decoder’s internal output (always referenced to 48 kHz) to the caller’s requested rate, and applies any cross-frame smoothing for mode transitions and redundancy (??).

The top-level dispatch is a compact `match` on the parsed TOC:

```
pub fn decode_packet(&mut self, data: &[u8]) -> Result<Vec<f32>,
PacketError> {
    if data.len() <= 1 {
        // Empty or TOC-only packet: conceal one frame (DTX / loss).
        return Ok(self.decode_lost(self.last_frame_size / self.ds));
    }
    let packet = Packet::parse(data)?;
    let toc = packet.toc();
    let mut out = Vec::new();
    for frame in packet.frames() {
        let pcm = self.decode_frame(
            frame, toc.mode(), toc.bandwidth(),
            toc.frame_size().samples_per_channel_48k(),
        );
        out.extend_from_slice(&pcm);
    }
    Ok(out)
}
```

2.4 Sample Rates and the 48 kHz Reference

The relationship between sample rates in Opus is a common source of confusion. The model is stated once here and referred to throughout.

The 48 kHz Reference Clock

Opus has three distinct notions of sample rate:

- The **audio bandwidth** (narrowband, mediumband, wideband, super-wideband, fullband) is a *property of the content*: how much of the spectrum carries signal. It is named in the TOC and caps the highest coded frequency (4, 6, 8, 12, or 20 kHz).
- The **internal rate** is the rate a coder actually runs at. SILK runs at 8, 12, or 16 kHz; CELT always runs at 48 kHz.
- The **API rate** is the rate the caller asks for (8, 12, 16, 24, or 48 kHz), at which PCM is delivered.

Crucially, **all timing is expressed in 48 kHz samples** regardless of the other two. A 20 ms frame is always 960 samples “at 48 kHz”; a 2.5 ms frame is 120. Granule positions in the Ogg container (??) count 48 kHz samples. The decoder resamples from the internal rate up to 48 kHz (CELT is already there), then *down* to the API rate by an integer factor $ds = 48000/f_s$. This single reference clock is what lets SILK frames, CELT frames, and the container agree on “when” a sample occurs even though they compute it at different rates.

2.5 From Quantities to Symbols

Every numeric quantity Opus transmits (a gain, an LSF index, a pulse vector, an energy delta) becomes a symbol in the range coder. The encoder and decoder share static probability models (*ICDF* tables, Section 3) for the structured fields and fall back to *raw bits* for the near-uniform ones. This arrangement has an important consequence:

Important Note

The bitstream is defined by the *sequence of range-coder symbols*, not by the floating-point intermediate values the encoder computed to choose them. Two encoders that pick the same symbols produce identical bytes, even if one used SIMD and `rsqrt` approximations and the other used scalar double-precision. This is why the decoder must be bit-exact (it is normative) but the encoder need only be *valid* (any choice the decoder can follow is legal). Most of ?? exploits exactly this freedom.

3 The Range Coder

Every bit Opus transmits passes through the range coder. Because the bitstream is defined entirely by the sequence of range-coder symbols, and because bit-exact conformance is verified through the coder's internal state (Section 3.7), it is the natural starting point and is treated here first and in detail.

3.1 Why Entropy Coding

Suppose a symbol s occurs with probability $p(s)$. Information theory says the *ideal* cost of transmitting it is its *self-information*, $-\log_2 p(s)$ bits [3, 1]. A common symbol (p near 1) should cost a fraction of a bit; a rare one, many bits. Over a long message drawn from a known distribution, the average cost approaches the *entropy* $H = -\sum_s p(s) \log_2 p(s)$, and no lossless code can do better.

A fixed-length code cannot reach this bound: it must spend a whole number of bits per symbol, wasting the fractional part. *Arithmetic coding*, and its integer-friendly cousin *range coding* [2, 6], do reach it, by abandoning the idea that each symbol maps to its own bit pattern. Instead, the *entire message* is encoded as a single number in $[0, 1)$, and each symbol narrows the interval that number must lie in.

Range Coding by Interval Subdivision

Begin with the interval $[0, 1)$. To encode a symbol s whose cumulative distribution carves out the sub-range $[fl/ft, fh/ft)$ (low and high cumulative frequencies out of a total ft), *replace* the current interval with that fraction of itself:

$$[\ell, \ell + r) \mapsto \left[\ell + r \frac{fl}{ft}, \ell + r \frac{fh}{ft} \right). \quad (1)$$

The interval shrinks by a factor of $(fh - fl)/ft = p(s)$ at each step, so after the whole message its width is $\prod_i p(s_i) = 2^{-\sum_i -\log_2 p(s_i)}$. Transmitting *any* number inside the final interval takes about $-\log_2(\text{width}) = \sum_i -\log_2 p(s_i)$ bits, the entropy. The decoder, knowing the same model, replays the subdivisions: it sees which sub-range the received number falls in, emits that symbol, rescales, and repeats.

The practical obstacle is that the interval bounds need ever more precision as they shrink, which is impossible with finite-width integers. The fix, due to Pasco and refined by Martin, is *renormalization*: as soon as the interval is narrow enough that its top bits are settled, *emit those bits* and shift them out, restoring headroom. Opus renormalizes a byte at a time.

3.2 The Integer Range Coder

Opus’s range coder (§4.1 for the decoder, §5.1 for the encoder) works in 32-bit integers. Its state is two words:

- **rng**: the current interval width, kept in $(2^{23}, 2^{31}]$.
- **val**: on the encoder, the interval’s low end; on the decoder, the offset of the received number from the top of the interval.

The governing constants name the renormalization threshold and the byte width:

```
const SYM_BITS:  u32 = 8;           // emit/consume one byte at a time
const SYM_MAX:   u32 = 255;         // 2^8 - 1
const CODE_BITS: u32 = 32;          // width of the (val, rng) state
const CODE_TOP:  u32 = 1 << 31;     // 2^31: initial encoder range
const CODE_BOT:  u32 = 1 << 23;     // 2^23: renormalize when rng
    falls to here
const CODE_SHIFT: u32 = 23;         // CODE_BITS - SYM_BITS - 1; carry
    bit position

#[inline]
const fn ilog(x: u32) -> u32 { u32::BITS - x.leading_zeros() } // 1 +
    floor(log2 x)
```

The lower bound $rng > 2^{23}$ guarantees that the division rng/ft in Equation (1) keeps enough significant bits for the largest total frequency Opus uses ($ft \leq 2^{16}$). Note `ilog` is just the hardware count-leading-zeros instruction; it appears everywhere the coder needs $\lfloor \log_2 \rfloor$.

3.2.1 Decoding a symbol

Decoding is two steps: `decode` reports *where* the received value sits in $[0, ft)$, the caller maps that to a symbol, then `update` shrinks the state to that symbol’s sub-range and renormalizes.

```
pub fn decode(&mut self, ft: u32) -> u32 {
    // Locate val within [0, ft): the symbol s satisfies fl <= decode()
    < fh.
    ft - (self.val / (self.rng / ft) + 1).min(ft)
}

pub fn update(&mut self, fl: u32, fh: u32, ft: u32) {
    let s = self.rng / ft;
    self.val -= s * (ft - fh);           // shift past the symbols
    above this one
    self.rng = if fl > 0 { s * (fh - fl) } // interior symbol: exact
    sub-width
    else { self.rng - s * (ft - fh) }; // first symbol
    keeps remainder
    self.normalize();
}
```

Two details are easy to miss and important to get right. First, the first symbol ($fl = 0$) is given the *remainder* of the range ($rng - s(ft - fh)$), not $s \cdot fh$; this is how the coder accounts for the truncation in $s = \lfloor rng/ft \rfloor$ and stays reversible. Second, `decode` caps its result at `ft` via `.min(ft)` to defend against a corrupt stream pushing the quotient out of range.

3.2.2 The renormalization loop

When `rng` drops to 2^{23} or below, the decoder shifts in a fresh byte:

```
fn normalize(&mut self) {
    while self.rng <= CODE_BOT {
        self.nbits_total += SYM_BITS;
        self.rng <= SYM_BITS;
        // Reassemble the next 8-bit symbol from the leftover bit of
        // the
        // previous byte (high) and the top 7 bits of the new byte (low
        // ).
        let byte = self.read_byte();
        let sym = (self.leftover_bit << 7) | (byte >> 1);
        self.leftover_bit = byte & 1;
        self.val = ((self.val << SYM_BITS) + (SYM_MAX - sym)) & 0
                    x7FFF_FFFF;
    }
}
```

The one-bit shimmy (`leftover_bit`) is a peculiarity of Opus's coder: the decoder works with a 7-bits-from-this-byte, 1-bit-carried representation so that the encoder's carry propagation (below) lands on byte boundaries. The `& 0x7FFF_FFFF` keeps `val` within 31 bits.

3.2.3 Encoding a symbol

The encoder mirrors the decoder exactly; it must, or the two will disagree and the stream will be undecodable:

```
pub fn encode(&mut self, fl: u32, fh: u32, ft: u32) {
    let r = self.rng / ft;
    if fl > 0 {
        self.val += self.rng - r * (ft - fl);
        self.rng = r * (fh - fl);
    } else {
        self.rng -= r * (ft - fh);
    }
    self.normalize();
}
```

The encoder's `normalize` emits the top byte of `val` instead of reading one, but is otherwise the same loop, with one complication that deserves its own treatment.

3.3 Carry Propagation

When the encoder adds to `val` (the `fl > 0` branch above), the addition can overflow into bits it has *already emitted*. A byte written out as `0xFF` might need to become `0x00` with a carry rippling into the byte before it. The encoder cannot un-write bytes, so it does not write them eagerly. Instead it buffers:

```

fn carry_out(&mut self, c: u32) {
    if c == SYM_MAX {
        self.ext += 1; // a run of 0xFF: defer, might
        all carry
    } else {
        let carry = (c >> SYM_BITS) as u8;
        if let Some(rem) = self.rem.take() {
            self.write_byte(rem + carry); // resolve the buffered
            byte
        }
        if self.ext > 0 {
            // Flush the deferred 0xFF run, +carry turns them into 0x00
            .
            let sym = (SYM_MAX + u32::from(carry)) as u8;
            for _ in 0..self.ext { self.write_byte(sym); }
            self.ext = 0;
        }
        self.rem = Some((c & SYM_MAX) as u8);
    }
}

```

Why a 0xFF Run Must Be Held

A carry into a 0xFF byte turns it into 0x00 and *propagates further left*. A carry into any other byte simply increments it and stops. So the encoder can safely emit a non-0xFF byte only once it knows whether a carry is coming, which it learns from the *next* byte. It therefore keeps one pending byte in `rem` and counts a run of 0xFFs in `ext`. When a non-0xFF byte arrives with carry c : the pending byte becomes $rem + c$, and the held 0xFFs become either 0xFF (no carry) or 0x00 (carry). This buffering is invisible to the decoder; it simply reads the bytes that finally emerge. The mechanism is what makes the encoder/decoder *symmetric to the bit*, which is the property the conformance oracle (Section 3.7) checks.

3.4 The ICDF Representation

Structured fields are coded against static probability tables. Opus stores them not as a PDF or a forward CDF but as an *inverse cumulative distribution function* (ICDF): for a model with total 2^{ftb} , the table holds $icdf[k] = 2^{ftb} - fh[k]$, the cumulative *tail* above each symbol, decreasing to a terminating zero.

Why Inverse CDFs

Two reasons. First, the decode primitive in Equation (1) naturally produces “how far down from the top is `val`”, which is exactly an upper-tail comparison, so an ICDF lets the decoder walk the table with one multiply and one compare per step and no running subtraction. Second, the tables are tiny (a handful of bytes), and storing the tail lets the common “most probable symbol first” ordering terminate early. The forward arithmetic (fl, fh) is recovered implicitly from adjacent ICDF entries.

```

pub fn decode_icdf(&mut self, icdf: &[u8], ftb: u32) -> usize {
    let d = self.val;
    let r = self.rng >> ftb;           // ft is a power of two: shift,
    // don't divide
    let mut k = 0;
    let mut t = self.rng;
    let mut s = r * u32::from(icdf[0]);
    while d < s {                       // walk down the tail until val
        sits in [s, t)
        k += 1;
        t = s;
        s = r * u32::from(icdf[k]);
    }
    self.val = d - s;                   // fold update() into the same
    // step
    self.rng = t - s;
    self.normalize();
    k
}

```

Because $ft = 2^{ftb}$ is a power of two, the division rng/ft becomes a shift, a small but pervasive saving, since ICDF decoding is the hottest path in SILK and in CELT's side information.

3.5 Binary Symbols and Raw Bits

Two specializations round out the coder.

Binary symbols with a power-of-two probability. Many flags have a probability of the form 2^{-logp} . These skip the table entirely:

```

pub fn decode_bit_logp(&mut self, logp: u32) -> bool {
    let s = self.rng >> logp;           // sub-range for the "1" outcome
    let bit = self.val < s;
    if !bit { self.val -= s; }
    self.rng = if bit { s } else { self.rng - s };
    self.normalize();
    bit
}

```

Raw bits. Some quantities are effectively uniform (a sign, the low bits of a pulse split), and modeling them as probabilities wastes effort. Opus codes these as *raw bits*, written at the *opposite end* of the same buffer from the arithmetic-coded data.

The Two-Ended Buffer

A range-coded packet has *two* write heads in one byte array. The arithmetic coder fills from the front, most-significant first, byte by byte as it renormalizes. Raw bits fill from the *back*, least-significant first. The two heads grow toward each other; the packet is well-formed as long as they do not cross. This costs nothing: raw bits need no probability model, so they need not interleave with the arithmetic stream, and it lets the finalizer (Section 3.6) pack the last arithmetic byte and the last raw byte into the same physical byte when they happen to fit. The decoder reads raw bits backwards from the end of the packet through a small 32-bit window, refilling a byte at a time.

```

pub fn decode_raw_bits(&mut self, bits: u32) -> u32 {
    if self.nend_bits < bits { // refill the
        window from the end
        loop {
            self.end_window |= self.read_byte_from_end() << self.
                nend_bits;
            self.nend_bits += SYM_BITS;
            if self.nend_bits > WINDOW_SIZE - SYM_BITS { break; }
        }
    }
    let ret = self.end_window & ((1 << bits) - 1); // LSB-first
    self.end_window >>= bits;
    self.nend_bits -= bits;
    ret
}

```

Uniform integers combine the two: a value in $[0, ft)$ with ft not a power of two is split into an arithmetic-coded high part (8 bits of model) and a raw-bit low part.

```

pub fn encode_uint(&mut self, t: u32, ft: u32) {
    let ftb = ilog(ft - 1);
    if ftb <= 8 {
        self.encode(t, t + 1, ft); // small: pure
        arithmetic
    } else {
        let ft_hi = ((ft - 1) >> (ftb - 8)) + 1;
        self.encode(t >> (ftb - 8), (t >> (ftb - 8)) + 1, ft_hi);
        self.encode_raw_bits(t & ((1 << (ftb - 8)) - 1), ftb - 8); //
        low bits raw
    }
}

```

3.6 Bit Budgeting: tell() and tell_frac()

Opus's bit allocation (??) needs to know, mid-frame, exactly how many bits have been spent so far, and it needs that figure to *agree* between encoder and decoder, since both run the same allocation arithmetic from the same running total. The coder exposes this through `tell` and its eighth-of-a-bit refinement `tell_frac`.

```

pub fn tell(&self) -> u32 { self.nbits_total - ilog(self.rng) }

pub(crate) fn tell_frac(nbits_total: u32, rng: u32) -> u32 {
    let mut lg = ilog(rng);
    let mut r = rng >> (lg - 16); // Q15 fractional part of log2(
    rng)
    for _ in 0..3 { // 3 iterations -> 1/8-bit
        resolution
        r = (r * r) >> 15;
        let bit = r >> 16;
        lg = 2 * lg + bit;
        r >>= bit;
    }
    nbits_total * 8 - lg
}

```

Bits Used: Total Shifted In Minus Range Headroom

The intuition: `nbits_total` counts every bit shifted into the coder. But the coder has not *committed* all of them: the current range `rng` still spans $\log_2(rng)$ bits of undecided value. So the bits actually consumed are `nbits_total` $- \log_2(rng)$. `tell()` uses the integer $\lfloor \log_2 \rfloor$ (a conservative whole-bit count); `tell_frac()` refines $\log_2(rng)$ to 3 fractional bits by the Newton-style squaring loop above, giving the 1/8-bit resolution Opus’s allocator works in. The two readouts agree on encoder and decoder because they are computed from the same $(nbits_total, rng)$.

3.7 The Final-Range Oracle

The range coder provides a single, powerful correctness check for any Opus implementation, described below.

rng-Agreement and the Final Range

After encoding (or decoding) the same sequence of symbols, the encoder and decoder must hold *identical* `rng` values, and, transitively, identical `val` and identical output bytes. RFC 6716 elevates this to a conformance tool: `OPUS_GET_FINAL_RANGE` returns the coder’s `rng` at the end of each packet, and a correct decoder’s final range must equal the reference encoder’s, *exactly*, for every packet.

This is a strong check. The entire decode (thousands of symbols across SILK and CELT, every table lookup, every renormalization) collapses into a single 32-bit number that is wrong if *any* step diverged. The `opus_native` conformance suite enforces it on all twelve official vectors: a passing decode is not merely close to the reference, it is arithmetically identical through the entropy coder. The PCM may differ by a rounding bit in the final float MDCT; the *range* may not differ at all.

In hybrid and redundancy cases (??) the packet carries more than one range-coded segment; the implementation XORs their final ranges together to form the single reported value, matching the reference’s bookkeeping.

4 Packet Framing

The range coder produces the bytes of a single frame. The packet layer (§3) is the envelope around those bytes: it names how to interpret the frame, allows several frames to share one packet, and defines the rules a decoder uses to reject malformed input. It contains no audio math and no entropy coding, only parsing.

4.1 The Self-Delimiting Question

A design decision shapes the entire packet format: an Opus packet is *not* self-delimiting. It does not record its own length. Opus assumes a transport layer (Ogg, RTP, WebRTC, Matroska) that already knows where each packet ends, and declines to duplicate that information.

Why Packets Carry No Length Field

Storing a length in every packet would waste two or three bytes on data the transport already has. At the low bitrates Opus targets, a 20 ms packet can be under 20 bytes, so a redundant length prefix is a 10–15% overhead. Opus instead infers frame boundaries from the total packet size supplied by the container. The cost is that the codec cannot be handed a bare byte stream and asked to find packet boundaries; it must be told, per packet, how many bytes belong to it. The one exception is the *self-delimiting* framing used inside multistream packets (??), where several Opus packets are concatenated and all but the last must announce their own length.

4.2 The TOC Byte

Every packet begins with one *Table of Contents* (TOC) byte. It packs three fields that, together, determine everything the decoder needs to dispatch the packet:

$$\underbrace{\text{c c c c c}}_{\text{config (5)}} \quad \underbrace{\text{s}}_{\text{stereo (1)}} \quad \underbrace{\text{n n}}_{\text{count code (2)}} \quad (2)$$

The top five bits select one of 32 *configurations*; the next bit is the mono/stereo flag; the low two bits are the frame-packing code (Section 4.3). The implementation treats the TOC as a transparent wrapper over the byte, with each field a small accessor:

```
pub struct Toc(u8);

impl Toc {
    pub const fn config(self) -> u8 { self.0 >> 3 } // bits 7..3
    pub const fn stereo(self) -> bool { (self.0 >> 2) & 1 == 1 } // bit 2
    pub const fn channels(self) -> u8 { 1 + ((self.0 >> 2) & 1) }
    pub const fn frame_count_code(self) -> u8 { self.0 & 0x3 } // bits 1..0
    pub const fn mode(self) -> Mode { /* from config */ }
    pub const fn bandwidth(self) -> Bandwidth { /* from config */ }
    pub const fn frame_size(self) -> FrameSize { /* from config */ }
}
```

4.2.1 The 32 configurations

The configuration number is not an arbitrary index; it is a structured encoding of (mode, bandwidth, frame size). The 32 values partition into the three modes, and within each mode the bandwidths and frame sizes are laid out in contiguous runs (Table 2). This regularity lets the encoder compute a config as a base value plus a frame-size offset, which is exactly how the encoder builds its TOC (??).

Three properties of Table 2 are worth noting. SILK-only modes support the long 40 and 60 ms frames that suit speech but never the short 2.5 and 5 ms frames; CELT-only modes are the reverse, supporting short low-delay frames down to 2.5 ms. Hybrid mode exists only at the two highest bandwidths, because it is defined as “SILK wideband plus a CELT extension”; below super-wideband there is no high band for CELT to add. And mediumband appears only for SILK, as CELT’s band layout has no 6 kHz cutoff.

4.3 Frame-Packing Codes

The two low bits of the TOC select one of four ways to pack frames into the remaining bytes. The motivation is amortization: a single packet can carry up to 120 ms of audio (for example three 40 ms

Config	Mode	Bandwidth	Frame sizes (ms)
0–3	SILK-only	narrowband (NB)	10, 20, 40, 60
4–7	SILK-only	mediumband (MB)	10, 20, 40, 60
8–11	SILK-only	wideband (WB)	10, 20, 40, 60
12–13	Hybrid	super-wideband (SWB)	10, 20
14–15	Hybrid	fullband (FB)	10, 20
16–19	CELT-only	narrowband (NB)	2.5, 5, 10, 20
20–23	CELT-only	wideband (WB)	2.5, 5, 10, 20
24–27	CELT-only	super-wideband (SWB)	2.5, 5, 10, 20
28–31	CELT-only	fullband (FB)	2.5, 5, 10, 20

Table 2: The 32 TOC configurations (§3.1, Table 2). The mode dictates which coder runs; the bandwidth caps the highest coded frequency; the frame size is always expressed at the 48 kHz reference.

frames), spreading the one-byte TOC overhead across multiple frames and reducing per-frame container overhead.

The Four Frame-Packing Codes

Let the bytes after the TOC be the *payload*. The count code determines how the payload splits into frames:

- **Code 0:** one frame, occupying the entire payload.
- **Code 1:** two frames of *equal* size; the payload length must be even, and each frame is half.
- **Code 2:** two frames of possibly *unequal* size; an explicit length precedes the first frame, and the second takes the rest.
- **Code 3:** an arbitrary count M of frames (1–48), with a following byte that selects constant-bitrate (CBR, all frames equal) or variable-bitrate (VBR, explicit lengths) packing, and optional padding.

Frame lengths, where coded explicitly (code 2 and VBR code 3), use a one- or two-byte representation: a byte below 252 is the length itself; a byte of 252 to 255 is combined with a following byte as $\text{second} \times 4 + \text{first}$, reaching up to 1275, the maximum frame size. A coded length of zero is legal and denotes a dropped or DTX (discontinuous transmission) frame that the decoder conceals.

The parser is a direct transcription of these four cases:

```
match toc.frame_count_code() {
  0 => { check_frame_len(rest.len())?; frames.push(rest); }
  1 => {
    if rest.len() % 2 != 0 { return Err(PacketError::
      Code1UnevenPayload); }
    let (a, b) = rest.split_at(rest.len() / 2);
    frames.push(a); frames.push(b);
  }
  2 => {
    let n1 = read_frame_len(&mut rest)?; // explicit first
    let length
    let frame1 = take(&mut rest, n1)?;
    frames.push(frame1); frames.push(rest); // second takes the
    remainder
  }
  _ => { /* code 3: count byte, optional padding, CBR or VBR lengths
    */ }
}
```

4.3.1 Code 3 in detail

Code 3 carries a second header byte after the TOC: bit 7 is the VBR flag, bit 6 signals the presence of padding, and the low six bits give the frame count M . Padding, when present, is a length encoded as a run of bytes (a value of 255 means “254 bytes of padding, continue reading”), allowing a packet to be inflated to a target size for constant-rate transport without affecting the audio. In VBR packing the header is followed by $M - 1$ explicit frame lengths, with the final frame taking the remainder; in CBR packing all M frames share the payload equally, and a payload not divisible by M is an error.

4.4 Validity Rules

RFC 6716 §3.4 enumerates seven conditions, labeled [R1] through [R7], that a well-formed packet must satisfy. They exist because a decoder fed hostile or corrupt input must reject it cleanly rather than read out of bounds or allocate unboundedly. The implementation maps each rule to a typed error (Table 3).

Rule	Condition	Error
R1	packet is at least one byte (has a TOC)	Empty
R2	every frame is at most 1275 bytes	FrameTooLarge
R3	code 1 payload length is even	Code1UnevenPayload
R4	coded frame lengths are not truncated	InvalidFrameLength
R5	code 3 count $M \geq 1$ and total ≤ 120 ms	InvalidFrameCount
R6	padding does not exceed the payload	InvalidPadding
R7	frame lengths do not overrun the payload	InvalidFrameLength

Table 3: The malformed-packet rules [R1]–[R7] (§3.4) and their errors.

Important Note

Rules R5 and R6 are the codec’s defense against denial-of-service input. R5 caps a packet at 120 ms of audio, so a single packet cannot demand an unbounded decode; the implementation checks $M \times (\text{tenths of ms per frame}) \leq 1200$. R6 ensures the padding count, which is attacker-controlled, cannot point past the end of the buffer. Differential fuzzing against the reference (over a million iterations) found two arithmetic-underflow bugs in earlier versions of the surrounding code, both since fixed by matching the reference’s signed semantics; the packet rules are the first line that stops such input from reaching the signal-processing layers at all.

5 CELT: Transform Coding

CELT, the Constrained-Energy Lapped Transform (§4.3), is the music half of Opus. It is a transform codec: it maps blocks of audio into the frequency domain with a Modified Discrete Cosine Transform, then spends bits on the resulting coefficients according to a psychoacoustic budget. Its distinguishing feature, the one its name advertises, is that it codes the *energy* of each frequency band explicitly and separately from the band’s *shape*, which keeps the reconstructed spectrum’s energy envelope faithful even when bits are scarce.

This section follows the decoder’s data flow: the transform (Section 5.1), the energy envelope (Section 5.2), the pyramid vector quantizer that codes band shapes (Section 5.3), the combinatorial enumeration that turns a quantized shape into an index (Section 5.4), the bit allocation that ties them together (??), the band-coding machinery (??), and finally the encoder’s analysis (??).

5.1 The MDCT and Lapped Transforms

A transform codec cannot simply cut the signal into blocks and transform each independently: the discontinuities at block boundaries would produce audible clicks, and the rectangular truncation would smear energy across frequencies (spectral leakage). The standard remedy is to overlap consecutive blocks and window them, but naive overlap doubles the data rate. The Modified Discrete Cosine Transform [princen1986analysis, princen1987subband] resolves this tension: it overlaps blocks by 50% yet remains *critically sampled*, producing exactly N coefficients for every N new input samples.

The Modified Discrete Cosine Transform

The MDCT maps $2N$ real input samples to N real coefficients:

$$X[k] = \sum_{n=0}^{2N-1} x[n] \cos \left[\frac{\pi}{N} \left(n + \frac{1}{2} + \frac{N}{2} \right) \left(k + \frac{1}{2} \right) \right], \quad k = 0, \dots, N-1. \quad (3)$$

The inverse (IMDCT) produces $2N$ samples from the N coefficients. Because the forward transform discards half its inputs' worth of degrees of freedom, a single IMDCT cannot recover its input; the missing information is supplied by the *overlapping* neighbor block. Each output block overlaps its successor by N samples, and the two contributions are added. This is ***time-domain aliasing cancellation*** (TDAC): the IMDCT of one block contains a time-reversed alias of the input, and the overlapping block's alias is equal and opposite, so the sum cancels it exactly.

5.1.1 The window and perfect reconstruction

TDAC cancels only if the analysis and synthesis windows satisfy the Princen-Bradley conditions. For a window $w[n]$ of length $2N$, perfect reconstruction requires

$$w[2N-1-n] = w[n] \quad (\text{symmetry}), \text{ and} \quad (4)$$

$$w[n]^2 + w[n+N]^2 = 1 \quad (\text{power complementarity}). \quad (5)$$

CELT uses the ***Vorbis window***, a power-complementary window built from a sine of a sine:

$$w[n] = \sin \left[\frac{\pi}{2} \sin^2 \left(\frac{\pi}{2L} \left(n + \frac{1}{2} \right) \right) \right], \quad n = 0, \dots, L-1, \quad (6)$$

where L is the overlap length. CELT does not window the full block: only the first and last L samples are tapered (with $L = 120$ at 48 kHz), and the interior is passed through unwindowed. This ***low-overlap*** design trades a little frequency selectivity for substantially reduced algorithmic delay, which is what lets Opus reach 5 ms latency in CELT mode.

TDAC Reconstruction in the IMDCT

The synthesis overlap-add is the kernel that makes the whole transform invertible. After the IMDCT produces the time-domain block, the overlap region is mixed with the tail held over from the previous frame, applying the window to both edges so that Equation (5) cancels the aliasing:

```
// Mirror both edges for TDAC, mixing with the previous frame's
// tail.
for i in 0..overlap / 2 {
    let (a, b) = (i, overlap - 1 - i);
    let (x1, x2) = (out[b], out[a]);
    let (w1, w2) = (window[i], window[overlap - 1 - i]);
    out[a] = w2 * x2 - w1 * x1;
    out[b] = w1 * x2 + w2 * x1;
}
```

The round trip (forward MDCT then inverse with overlap-add) is verified by a perfect-reconstruction test: a windowed block transformed and inverted returns the input to floating-point precision, with zero delay and unit gain.

5.1.2 Short blocks for transients

A long MDCT gives fine frequency resolution but coarse time resolution, which smears a sharp transient (a drum hit, a consonant) backward in time as *pre-echo*: quantization noise spread across the whole block becomes audible before the attack that produced it. CELT’s defense is to switch, for transient frames, from one long MDCT to a series of B short MDCTs (with $B = 2, 4, 8$), each covering a fraction of the frame. The encoder signals this choice with a transient flag (??); the short transforms’ coefficients are then interleaved so the band structure still applies. Time resolution improves by a factor of B at the cost of frequency resolution, exactly the uncertainty trade-off, applied adaptively per frame.

5.1.3 Computing the MDCT through an FFT

Evaluated directly, Equation (3) costs $O(N^2)$. In practice the MDCT factors into three stages: a complex *pre-rotation* that folds the $2N$ real inputs into $N/4$ complex values, an $N/4$ -point complex FFT, and a complex *post-rotation* that unpacks the result. The FFT dominates the cost and is isolated behind a seam in the implementation: the default build routes it through an external planned-FFT backend, while a dependency-free build falls back to a direct $O(N^2)$ evaluation. The numerical and performance consequences of this seam are treated in ??; here it suffices that the transform is an $O(N \log N)$ operation in the production configuration.

5.2 The Energy Envelope

CELT divides the MDCT spectrum into 21 *bands* whose widths follow the ear’s critical bands: narrow at low frequencies where the ear resolves pitch finely, wide at high frequencies where it does not [zwicker1999psychoacoustics]. For each band, CELT codes two things separately: the band’s *energy* (the L^2 norm of its coefficients) and its *shape* (the unit-norm direction). This separation is the “Constrained-Energy” in the codec’s name.

Why Code Energy Separately

The audibility of a frequency band is governed largely by how much energy it contains, not by the precise arrangement of coefficients within it. By coding the energy envelope explicitly and protecting it with its own bits, CELT guarantees that the reconstructed signal has the right spectral balance even when the shape is coded coarsely. A codec that quantized raw coefficients would instead let energy drift as bits run short, producing audible spectral tilt and “birdie” artifacts. The cost of the guarantee is the overhead of the envelope itself, which CELT minimizes with aggressive prediction.

Energy is coded in the $\log_2$ domain in three passes of decreasing significance:

1. **Coarse energy** quantizes each band’s log-energy to roughly 6 dB resolution, predicted from neighbors and coded with a Laplace model.
2. **Fine energy** refines each band with a number of raw bits set by the allocation, halving the error per bit.
3. **Final bits** distribute any leftover budget, one bit at a time, to the bands that the allocation flagged as highest priority.

5.2.1 Coarse energy and two-dimensional prediction

The coarse energy of a band is highly predictable from two neighbors: the same band in the previous frame (time prediction) and the adjacent band in the same frame (frequency prediction). CELT codes only the prediction residual, with a Laplace probability model whose width depends on the frame size. The update applies a prediction coefficient α in time and accumulates a running frequency prediction with a leakage factor β :

$$E_i \leftarrow \alpha E_i^{\text{prev}} + p_i + q_i, \quad p_{i+1} = p_i + q_i(1 - \beta), \quad (7)$$

where q_i is the decoded residual for band i and p_i the frequency predictor carried across bands. Intra frames (no time prediction) set $\alpha = 0$ and use a larger β .

```
let (coef, beta) = if intra { (0.0, BETA_INTRA) }
                    else    { (PRED_COEF[lm], BETA_COEF[lm]) };
let mut prev = [0.0f32; 2];
for i in start..end {
    for (c, prev_c) in prev.iter_mut().enumerate().take(channels) {
        let qi = if budget - tell >= 15 {
            ec_laplace_decode(dec, /* model for band i, frame size lm
            */)
        } else { /* budget-starved fallback code */ };
        let old = state.old_ebands[c][i].max(-9.0);
        state.old_ebands[c][i] = coef * old + *prev_c + qi as f32;
        *prev_c += qi as f32 - beta * qi as f32;
    }
}
```

The Laplace coder (§4.3.2.1) is a specialization of the range coder for a two-sided geometric distribution: small residuals near zero are cheap, large ones progressively more expensive, matching the statistics of inter-frame energy deltas. When the remaining budget falls below the threshold to afford a Laplace symbol, the coder drops to a fixed fallback code so the decode never stalls.

5.3 Pyramid Vector Quantization

Once a band’s energy is known, its shape is a unit vector in N dimensions (for a band of N coefficients). CELT quantizes that direction with *pyramid vector quantization* (PVQ) [fischer1986pyramid],

a structured vector quantizer with a property that makes it ideal here: every codeword has exactly the same norm, so quantizing the shape never disturbs the separately coded energy.

The Pyramid Codebook

The PVQ codebook of dimension N and *pulse count* K is the set of all integer vectors $\mathbf{y} \in \mathbb{Z}^N$ whose absolute components sum to K :

$$S(N, K) = \left\{ \mathbf{y} \in \mathbb{Z}^N : \sum_{i=0}^{N-1} |y_i| = K \right\}. \quad (8)$$

Geometrically these are the lattice points on the surface of an N -dimensional L^1 ball, the “pyramid.” A band’s shape is coded by finding the codeword $\mathbf{y}$ closest in direction to the unit shape, then transmitting which codeword it is. On decode, $\mathbf{y}$ is read back and normalized to unit norm, $\hat{\mathbf{x}} = \mathbf{y}/\|\mathbf{y}\|$, and scaled by the band’s energy. The pulse count K controls the resolution: more pulses mean a denser codebook and a finer shape, and K is exactly what the bit allocation (??) decides per band.

5.3.1 The pulse search

Finding the best codeword is a search over $S(N, K)$, which would be enormous to enumerate, so CELT uses a greedy algorithm that adds pulses one at a time, each time placing a pulse in the position that most improves the correlation between the candidate $\mathbf{y}$ and the target shape $\mathbf{x}$. After each pulse, the quantity maximized is

$$j^* = \arg \max_j \frac{(\mathbf{x} \cdot \mathbf{y} + x_j)^2}{\|\mathbf{y}\|^2 + 2y_j + 1}, \quad (9)$$

the squared cosine similarity that would result from adding a pulse at position j . The algorithm starts from a projection of $\mathbf{x}$ onto the pyramid to place most of the pulses cheaply, then refines pulse by pulse. The inner loop of Equation (9) is the single hottest computation in CELT encoding and is the subject of a dedicated SIMD kernel (??).

5.4 CWRs: Enumerating the Codebook

The PVQ search yields a codeword $\mathbf{y}$; the coder must transmit *which* of the $V(N, K) = |S(N, K)|$ codewords it is, as a uniform integer in $[0, V(N, K))$. This is the **combinatorial** step: a bijection between pulse vectors and indices, known in the Opus sources as CWRs (coding with replacement and signs, §4.3.4.2).

Counting Pulse Vectors

Let $V(N, K)$ be the number of length- N integer vectors with K signed pulses, and let $U(N, K)$ count those whose first nonzero entry is positive. These satisfy the recurrences

$$U(N, K) = U(N-1, K) + U(N, K-1) + U(N-1, K-1), \quad (10)$$

$$V(N, K) = U(N, K) + U(N, K+1), \quad (11)$$

with $U(0, 0) = 1$. The three terms of Equation (10) correspond to the first position holding zero, holding a pulse with more to come, or holding a pulse that completes the budget. The index of a particular $\mathbf{y}$ is built by walking the positions and, at each, adding the count of all codewords that sort before the chosen one, the standard combinatorial-number-system ranking.

Computing U from Equation (10) on every band decode would cost $O(NK)$. The implementation instead precomputes the U rows into a flat table once, so ranking and unranking a codeword become

$O(N)$ table lookups. The table is small (a few hundred entries, generated on first use) and exploits the symmetry $U(n, k) = U(k, n)$ to stay compact:

```
// Build the U-table once: each entry is the recurrence of Eq. (cwrs_u)
.
fn pvq_u_data() -> &'static [u32] {
    static T: OnceLock<Vec<u32>> = OnceLock::new();
    T.get_or_init(|| {
        let mut uu = [[0u32; 177]; 15];
        uu[0][0] = 1;
        for a in 1..15 {
            for b in 1..177 {
                uu[a][b] = uu[a - 1][b]
                    .wrapping_add(uu[a][b - 1])
                    .wrapping_add(uu[a - 1][b - 1]);
            }
        }
        /* flatten into a row-offset table */
        flatten(uu)
    })
}
```

The decoder reads the uniform index from the range coder, unranks it to recover $\mathbf{y}$ through the table, and normalizes. Because the index is uniform over exactly $V(N, K)$ values, no bits are wasted: the shape costs precisely $\log_2 V(N, K)$ bits, which is what the allocator budgeted for.

5.5 Bit Allocation

The energy envelope and the PVQ shapes both consume bits, and the number of pulses K available to each band is whatever the allocation leaves for it. The allocation is the arbiter that converts a frame's total bit budget into a per-band pulse count. Its defining constraint is subtle and important.

The Allocation Must Be Implicit

The decoder needs to know each band's pulse count *before* it can decode that band's shape, but the pulse counts are never transmitted: there are no bits to spare for them. Instead, encoder and decoder run the *same* deterministic allocation procedure over the same inputs (the total budget, the frame size, static tables, and a few coded parameters), and arrive at identical per-band allocations. This is why the bit budget is tracked to 1/8-bit precision through `tell_frac` (Section 3.6): the allocation arithmetic must produce bit-identical results on both sides, or the decoder will read a band with the wrong pulse count and desynchronize from the range coder. The allocation is, in effect, a pseudo-random function of the budget that both parties compute independently.

The procedure (§4.3.3) works in units of 1/8 bit (the constant `BITRES = 3`). It begins from a family of static **allocation vectors**, tabulated target bit counts per band for a range of quality levels, and interpolates between two adjacent vectors to match the available budget. The interpolation fraction is found by bisection: at each step it tentatively allocates at the midpoint, sums the cost over all bands (clamped to per-band caps, with bands below a floor skipped), and moves the bisection bound according to whether the total overran the budget.

```

// Bisect the interpolation fraction in 1/64 steps to fit 'total'.
let (mut lo, mut hi) = (0i32, 1 << ALLOC_STEPS);
for _ in 0..ALLOC_STEPS {
    let mid = (lo + hi) >> 1;
    let mut psum = 0i32;
    let mut done = false;
    for j in (start..end).rev() {
        let tmp = bits1[j] + ((mid * bits2[j]) >> ALLOC_STEPS);
        if tmp >= thresh[j] || done { done = true; psum += tmp.min(cap[
            j]); }
        else if tmp >= alloc_floor { psum += alloc_floor; }
    }
    if psum > total { hi = mid; } else { lo = mid; }
}

```

Once the per-band bit targets are fixed, each band’s allocation is split between *fine-energy* bits (raw bits that refine the envelope) and *shape* bits (the PVQ codeword). The conversion between a bit budget and a pulse count K goes through a precomputed *pulse cache*: `bits2pulses` binary- searches the cache for the largest K whose codeword fits the budget, and `pulses2bits` reports the exact cost $\lceil \log_2 V(N, K) \rceil$ of a given K . Surplus bits left after the split, the *balance*, are carried forward to subsequent bands so nothing is wasted. The allocation also decides, from the budget, the boundary band for intensity stereo and whether to use dual stereo (??), and it may explicitly *skip* high bands when the budget cannot support them, coding the skip decisions as run-length flags.

5.6 Band Coding: Splits, Folding, and Stereo

With energy and allocation settled, the band loop codes each band’s shape. A band of N coefficients is rarely quantized as one PVQ vector; instead it is recursively halved, with the split coded by an angle, until the pieces are small enough to quantize directly. Several refinements layer on top: spreading to decorrelate noisy bands, folding to fill empty ones, anti-collapse to repair transient artifacts, and stereo coupling.

5.6.1 Recursive splitting by angle

When a band is split in two, the energy is divided between the halves by an angle θ : the lower half receives $\cos \theta$ of the amplitude and the upper $\sin \theta$. Quantizing θ (rather than two independent energies) keeps the total band energy exactly fixed, preserving the energy constraint through the recursion. The number of quantization levels for θ depends on the band’s bit allocation, and the angle is coded with one of three probability models chosen by context (§4.3.4.1): a *uniform* PDF for ordinary time splits, a *triangular* PDF that biases toward an even split for mono bands, and a *step* PDF for the first stereo split. The recursion terminates when a sub- band is small enough, at which point it is coded by a single PVQ vector (Section 5.3).

5.6.2 Spreading rotation

Concentrating K pulses in N positions produces a sparse, spiky vector, which in a band that should sound noise-like is audible as tonal “birdies.” CELT counters this by applying a *spreading rotation*: a sequence of Givens rotations that smear the pulses’ energy across the band without changing its norm. The amount of spreading (none, light, normal, or aggressive) is chosen by the encoder’s tonality analysis (??) and signaled once per frame.

5.6.3 Folding and noise fill

When the allocation gives a band zero pulses, leaving it empty would create silent spectral holes. Instead CELT fills the band by **folding**: copying already-decoded coefficients from lower frequencies into the empty band, then renormalizing to the band’s coded energy. When no suitable lower content exists, the band is filled with pseudo-random noise from a small linear congruential generator. The generator and the fold share the same role, supplying a plausible shape at the correct energy for bands too cheap to code explicitly.

```
// Linear congruential generator used for noise fill (Numerical Recipes
    constants).
fn celt_lcg_rand(seed: u32) -> u32 {
    seed.wrapping_mul(1_664_525).wrapping_add(1_013_904_223)
}
```

5.6.4 Collapse masks and anti-collapse

In a transient frame coded as B short blocks (Section 5.1), a band’s pulses may all land in a single short block, leaving the others at zero, a **collapse**. A collapsed block is silent for part of the frame, which can sound like a dropout. CELT tracks a **collapse mask**, one bit per short block per band, recording which blocks received energy. The **anti-collapse** step then injects a controlled amount of noise into the collapsed blocks, scaled to the minimum of the neighboring bands’ energies, so every short block carries at least a floor of energy. The presence of anti-collapse is controlled by a single coded bit per transient frame.

5.6.5 Stereo coupling

For stereo input, CELT codes each band as a *mid* (sum) and *side* (difference) pair rather than left and right, because the mid/side representation concentrates energy in the mid channel and codes the stereo image efficiently. Three sub-modes apply, chosen per band by the allocation:

- **Mid/side with angle:** a stereo angle θ apportions energy between mid and side, coded like the mono split above.
- **Intensity stereo:** above a boundary band set by the budget, only the mid is coded and the side is reconstructed by copying the mid, discarding stereo detail where it is least audible (at high frequencies, low bitrates).
- **Dual stereo:** the two channels are coded fully independently, used when the channels are sufficiently decorrelated to justify the bits.

A special case handles bands of $N = 2$, where a single angle suffices to code the stereo relationship directly.

5.6.6 Time-frequency reshaping

Within a frame, individual bands can opt for finer time resolution (at the expense of frequency resolution) or vice versa, independently of the global short-block decision. This is implemented with in-place Haar and Hadamard transforms that redistribute a band’s coefficients between time and frequency before quantization. A per-frame `tf_select` flag and per-band flags encode the choice, which the encoder optimizes by the analysis below.

5.7 The Encoder’s Analysis

Everything to this point is shared by encoder and decoder; the decoder simply reads the parameters and follows. The encoder must *decide* those parameters, and those decisions, not the bitstream format, are where encoders differ in quality. CELT’s encoder runs several analyses per frame.

Transient detection. The encoder decides whether to use short blocks by running the input through a high-pass filter and then forward and backward energy followers; a sharp rise in the high-passed energy relative to its smoothed envelope marks a transient. The high-pass is a second-order section, and its output drives the masking metric:

```
// High-pass filter: (1 - 2z^-1 + z^-2) / (1 - z^-1 + 0.5 z^-2).
let (mut mem0, mut mem1) = (0.0f32, 0.0f32);
for (t, &x) in tmp.iter_mut().zip(input.iter()) {
    let y = mem0 + x;
    let mem00 = mem0;
    mem0 = mem0 - x + 0.5 * mem1;
    mem1 = x - mem00;
    *t = y;
}
```

Time-frequency analysis. For the bands, the encoder chooses time-versus- frequency resolution by an L^1 -metric minimization solved with a small Viterbi search (`tf_analysis` and `tf_select`), trading the cost of changing resolution against the coding gain it yields band by band.

Spreading decision. A tonality measure over the spectrum selects the spreading amount (??): tonal frames get little or no spreading, noisy frames get aggressive spreading.

Dynamic allocation and trim. Two analyses shape the allocation before it runs. *Dynamic allocation* (`dynalloc_analysis`) computes per-band importance boosts from a masking model, steering extra bits to perceptually important bands. The *allocation trim* (`alloc_trim_analysis`) sets a single global parameter that tilts the budget toward low or high frequencies based on spectral tilt and transientness.

The pitch pre-filter. Before the MDCT, the encoder optionally applies a comb *pre-filter* that whitens the signal’s harmonic structure, the exact inverse of the comb post-filter the decoder applies after synthesis. Whitening the harmonics concentrates energy and improves coding efficiency on strongly tonal material. The pitch period and gain that parameterize the filter come from a pitch analysis (downsampling, decimated cross-correlation, and octave-error rejection) that mirrors the technique used for packet-loss concealment (??).

6 SILK: Linear-Predictive Coding

SILK (§4.2) is the speech half of Opus. Where CELT works in the frequency domain on arbitrary signals, SILK works in the time domain on a *model* of how speech is produced, and codes only what the model fails to predict. This model-based approach is what lets SILK code intelligible speech at a few kilobits per second, far below what a general transform coder achieves. SILK runs at one of three internal rates (8, 12, or 16 kHz, for narrowband, mediumband, and wideband), in frames of 10, 20, 40, or 60 ms divided into 2 or 4 subframes.

This section develops the model bottom-up: the source-filter idea (??), the short-term predictor and how it is estimated and represented (????), the long-term predictor that captures pitch (??), the gains and excitation coding (????), the noise-shaping quantizer at the encoder’s heart (??), and the supporting machinery of stereo, resampling, and control (??????).

6.1 The Source-Filter Model

Speech as Source and Filter

Human speech is produced by an excitation source (the vocal folds, for voiced sounds, or turbulent airflow, for unvoiced ones) passing through a filter (the vocal tract, whose shape determines the formants we hear as vowels). Linear-predictive coding [makhoul1975linear, rabiner1978digital] models this directly: the vocal tract becomes an all-pole filter $1/A(z)$, and the excitation becomes a sparse residual signal. The model predicts each sample from a weighted sum of recent samples,

$$\hat{s}[n] = - \sum_{k=1}^p a_k s[n-k], \quad (12)$$

and codes only the residual $e[n] = s[n] - \hat{s}[n]$ that the prediction misses. Because the residual carries far less energy and structure than the original, it is cheap to code. SILK adds a second predictor for the residual's own periodicity (pitch) and a noise-shaping loop that hides quantization noise under the speech spectrum.

The frame is the unit at which SILK transmits its model; the subframe is the unit at which the model is applied. Each frame carries one set of short-term filter coefficients (interpolated across subframes for 20 ms frames), and per-subframe gains and pitch parameters. The decoder reconstructs each frame by running the excitation through the long-term and short-term synthesis filters, then resampling to the output rate.

6.2 Short-Term Prediction

The short-term predictor of ?? is a p -tap linear filter ($p = 10$ for narrowband and mediumband, $p = 16$ for wideband). It comes in two forms that are exact inverses: the *analysis* filter $A(z)$, which the encoder applies to produce the residual, and the *synthesis* filter $1/A(z)$, which the decoder applies to reconstruct the signal.

```
// Analysis: e[n] = x[n] + sum_k a_k x[n-1-k] (apply A(z))
pub fn lpc_residual(samples: &[f32], coeffs: &LpcCoefficients) -> Vec<
    f32> {
    /* residual[n] = samples[n] + sum coeffs[k] * samples[n-1-k] */
}

// Synthesis: y[n] = e[n] - sum_k a_k y[n-1-k] (apply 1/A(z))
pub fn lpc_synthesis(residual: &[f32], coeffs: &LpcCoefficients) -> Vec<
    f32> {
    /* output[n] = residual[n] - sum coeffs[k] * output[n-1-k] */
}
```

6.2.1 Estimating the coefficients

The decoder receives the coefficients; the encoder must estimate them from the signal. Two classical methods apply. *Levinson-Durbin* recursion [levinson1947wiener] solves the normal equations relating the filter coefficients to the signal's autocorrelation, in $O(p^2)$ time, producing the minimum mean-square prediction error. *Burg's method* [burg1975maximum] instead estimates reflection coefficients directly from the signal by minimizing the sum of forward and backward prediction errors, which behaves better on the short analysis windows speech coding uses. The SILK encoder uses a modified Burg method, and caps the resulting filter's prediction gain to keep it numerically well-conditioned.

Levinson-Durbin Recursion

The recursion builds the order- p predictor from the order- $(p-1)$ predictor by computing a reflection coefficient k_m that drives the next prediction error to its minimum, updating all coefficients in place:

```
pub fn levinson_durbin(autocorr: &[f64], order: usize) -> Vec<f64>
{
    let mut a = vec![0.0; order + 1];
    let mut err = autocorr[0];
    for m in 1..=order {
        let mut k = autocorr[m];
        for i in 1..m { k += a[i] * autocorr[m - i]; }
        k = -k / err; // reflection coefficient
        a[m] = k;
        for i in 1..=m / 2 { // symmetric in-place update
            let tmp = a[i] + k * a[m - i];
            a[m - i] += k * a[i];
            a[i] = tmp;
        }
        err *= 1.0 - k * k; // error shrinks each order
    }
    a
}
```

The factor $1 - k_m^2$ is non-negative exactly when $|k_m| \leq 1$, the condition for filter stability, which is why reflection coefficients are a convenient parameterization.

6.3 Line Spectral Frequencies

The encoder cannot transmit LPC coefficients directly: they are extremely sensitive to quantization, and a small rounding error can push the all-pole synthesis filter outside the unit circle, making it unstable and the decoded signal explode. SILK transmits the filter in a different representation, the *line spectral frequencies* (LSFs), which is built for quantization.

Why Line Spectral Frequencies

The order- p polynomial $A(z)$ is decomposed into a symmetric and an antisymmetric polynomial,

$$P(z) = A(z) + z^{-(p+1)} A(z^{-1}), \quad (13)$$

$$Q(z) = A(z) - z^{-(p+1)} A(z^{-1}), \quad (14)$$

whose roots, the LSFs, all lie *on* the unit circle and *interlace* [itakura1975line]. This representation has three properties that make it ideal for coding. First, stability is trivially checked and enforced: as long as the LSFs stay ordered, the reconstructed filter is stable. Second, the LSFs are localized in frequency, so each one affects the spectrum mainly near its own frequency, and a quantization error in one LSF stays local rather than corrupting the whole spectrum. Third, they interpolate gracefully, so SILK can transmit one set per frame and interpolate across subframes. SILK uses *normalized* LSFs (NLSFs), scaled so the frequency range maps to $[0, 1)$ and stored as Q15 integers.

6.3.1 The two-stage vector quantizer

NLSFs are coded with a two-stage vector quantizer (§4.2.7.5). The first stage selects one of 32 codebook vectors that approximates the whole NLSF vector, coded with an entropy model. The second stage codes the per-coefficient residual between the true NLSFs and the first-stage approximation, using a predictive scheme (each residual partly predicted from the previous) and an arithmetic-coded index. The residual quantization is weighted by *Laroia weights* [laroia1991robust], which approximate each coefficient’s perceptual sensitivity (inversely related to its spacing from its neighbors), so bits are spent where the ear notices. The encoder searches this quantizer with a delayed- decision trellis, keeping several candidate paths alive to find a low-rate, low-distortion combination.

6.3.2 Reconstruction and stability enforcement

The decoder reverses the process: dequantize the residual, add the first-stage vector, enforce a minimum spacing between adjacent NLSFs to guarantee stability, and convert back to LPC coefficients. The conversion (`nlsf2a`) builds the P and Q polynomials from the cosines of the NLSFs and combines them. Even after stabilization, the resulting filter is checked for stability, and if it fails, bandwidth expansion (scaling the coefficients toward zero) is applied until it passes:

```
// After converting NLSFs to LPC, force stability by bandwidth
// expansion.
while lpc_inverse_pred_gain(&a_q12[..d]) == 0 && i <
    MAX_LPC_STABILIZE_ITERATIONS {
    bwexpander_32(&mut a32_qa1[..d], 65536 - (2 << i)); // shrink
    toward zero
    /* re-quantize a_q12 from a32_qa1 and re-test */
    i += 1;
}
```

For 20 ms frames, the decoder interpolates between the previous frame’s NLSFs and the current frame’s, using a 2-bit interpolation factor, so the filter evolves smoothly across the frame’s first and second halves rather than jumping at the midpoint.

6.4 Long-Term Prediction

Short-term prediction removes the spectral envelope, but the residual of voiced speech is still strongly periodic at the pitch period: the vocal folds produce a pulse train, and successive pitch periods resemble one another. SILK removes this redundancy with a second predictor, the *long-term predictor* (LTP), which predicts the current residual from the residual one pitch period earlier.

Pitch Prediction with a Five-Tap Filter

For a voiced subframe with pitch lag T (in samples), the LTP predicts the residual $e[n]$ from a five-tap filter applied around $e[n - T]$:

$$\hat{e}[n] = \sum_{j=-2}^2 b_j e[n - T + j]. \quad (15)$$

The five taps b_j let the predictor track a pitch period that does not fall exactly on a sample boundary and that changes shape slowly from period to period. What remains after long-term prediction, the doubly-predicted residual, is the sparse excitation that SILK actually codes. The pitch lag T and the taps b_j are themselves transmitted: the lag as a per-frame absolute value plus per-subframe offsets from a contour codebook, and the taps as an index into one of three gain codebooks.

The lag is decoded as a base lag plus a per-subframe contour offset, clamped to the valid pitch range for the internal rate:

```
let (min_lag, max_lag) = (PE_MIN_LAG_MS * fs_khz, PE_MAX_LAG_MS *
    fs_khz);
let lag = min_lag + i32::from(lag_index);
for (k, out) in pitch_lags.iter_mut().enumerate().take(nb_subfr) {
    *out = (lag + contour_offset(k)).clamp(min_lag, max_lag); // per-
        subframe lag
}
```

The five-tap gains come from one of three codebooks of 8, 16, or 32 vectors, selected by a per-frame *periodicity index* that trades codebook size against the bits to address it: highly periodic speech justifies the larger, more precise codebook. The encoder chooses the lag with a three-stage pitch search (??) and the gains with a rate-distortion-weighted vector quantizer.

7 Python API Reference

7.1 Introduction

Python bindings for `opus_native` are planned but not yet released. This section is reserved for the complete Python reference — installation, the encoder and decoder objects, streaming and file (Ogg Opus) helpers, and worked examples that apply the concepts from the preceding sections in practice.

Important Note

This chapter is a placeholder. The bindings (a PyO3 extension module mirroring the Rust `OpusEncoder/OpusDecoder` surface, with NumPy buffer-protocol interop for PCM) are under construction. Until they ship, use the Rust API documented in ?? directly, or call the C reference `libopus` through an existing binding. The remainder of this section is intentionally left blank and will be filled in alongside the first binding release.

Installation and Imports

Decoding

Encoding

Streaming and Files

Complete API Reference

References

- [1] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. 2nd. Wiley-Interscience, 2006.
- [2] G. Nigel N. Martin. “Range Encoding: An Algorithm for Removing Redundancy from a Digitised Message”. In: *Video and Data Recording Conference, Southampton* (1979).
- [3] Claude E. Shannon. “A Mathematical Theory of Communication”. In: *The Bell System Technical Journal* 27.3 (1948), pp. 379–423. DOI: [10.1002/j.1538-7305.1948.tb01338.x](https://doi.org/10.1002/j.1538-7305.1948.tb01338.x).
- [4] Jean-Marc Valin and Timothy B. Terriberry. *Updates to the Opus Audio Codec*. Request for Comments RFC 8251. Internet Engineering Task Force (IETF), Oct. 2017. DOI: [10.17487/RFC8251](https://doi.org/10.17487/RFC8251). URL: <https://www.rfc-editor.org/rfc/rfc8251>.
- [5] Jean-Marc Valin, Koen Vos, and Timothy B. Terriberry. *Definition of the Opus Audio Codec*. Request for Comments RFC 6716. Internet Engineering Task Force (IETF), Sept. 2012. DOI: [10.17487/RFC6716](https://doi.org/10.17487/RFC6716). URL: <https://www.rfc-editor.org/rfc/rfc6716>.
- [6] Ian H. Witten, Radford M. Neal, and John G. Cleary. “Arithmetic Coding for Data Compression”. In: *Communications of the ACM* 30.6 (1987), pp. 520–540. DOI: [10.1145/214762.214771](https://doi.org/10.1145/214762.214771).